Defined in active_support/core_ext/range/overlaps.rb.

14 Extensions to Proc

14.1 bind

As you surely know Ruby has an UnboundMethod class whose instances are methods that belong to the limbo of methods without a self. The method Module#instance_method returns an unbound method for example:

```
Hash.instance_method(:delete) # => #<UnboundMethod: Hash#delete>
```

An unbound method is not callable as is, you need to bind it first to an object with bind:

```
clear = Hash.instance_method(:clear)
clear.bind({a: 1}).call # => {}
```

Active Support defines Proc#bind with an analogous purpose:

```
Proc.new { size }.bind([]).call # => 0
```

As you see that's callable and bound to the argument, the return value is indeed a Method.

To do so Proc#bind actually creates a method under the hood. If you ever see a method with a weird name like __bind_1256598120_237302 in a stack trace you know now where it comes from.

Action Pack uses this trick in rescue_from for example, which accepts the name of a method and also a proc as callbacks for a given rescued exception. It has to call them in either case, so a bound method is returned by handler_for_rescue, thus simplifying the code in the caller:

```
def handler_for_rescue(exception)
  _, rescuer = Array(rescue_handlers).reverse.detect do |klass_name,
handler|
    ...
  end

  case rescuer
  when Symbol
    method(rescuer)
  when Proc
    rescuer.bind(self)
  end
end
```

Defined in active_support/core_ext/proc.rb.

15 Extensions to Date

15.1 Calculations

All the following methods are defined in active_support/core_ext/date/calculations.rb.

The following calculation methods have edge cases in October 1582, since days 5..14 just do not exist. This guide does not document their behavior around those days for brevity, but it is enough to say that they do what you would expect. That is, Date.new(1582, 10, 4).tomorrow returns Date.new(1582, 10, 15) and so on. Please check test/core_ext/date_ext_test.rb in the Active Support test suite for expected behavior.

15.1.1 Date.current

Active Support defines Date.current to be today in the current time zone. That's like Date.today, except that it honors the user time zone, if defined. It also defines Date.yesterday and Date.tomorrow, and the instance predicates past?, today?, and future?, all of them relative to Date.current.

When making Date comparisons using methods which honor the user time zone, make sure to use Date.current and not Date.today. There are cases where the user time zone might be in the future